EM algorithm for GMM

Annotated implementation with the formulas used in class

AUTHOR
Patricia Menéndez, Matthew Stephens
and Heejung Shim

Simulate data

Example 1: well-separated components

Simulate 1000 samples from the following mixture model. For $i = 1, \ldots, 1000$,

$$Z_i \sim \text{Categorical}(0.2, 0.3, 0.5),$$

and

$$\begin{aligned} X_i \mid Z_i = 1 &\sim N(\mu_1 = -10, \sigma_1^2 = 2), \\ X_i \mid Z_i = 2 &\sim N(\mu_2 = 0, \sigma_2^2 = 2), \\ X_i \mid Z_i = 3 &\sim N(\mu_3 = 6, \sigma_3^2 = 2). \end{aligned}$$

Recall the generative interpretation of a mixture model. For each observation we first generate the hidden label Z_i, and then generate X_i from the Normal distribution corresponding to that label.

Equivalently,

$$P(Z_i = k) = \pi_k,$$

and

$$X_i \mid Z_i = k \sim N(\mu_k, \sigma_k^2).$$

Here the true parameters are

$$(\pi_1, \pi_2, \pi_3) = (0.2, 0.3, 0.5),$$

$$(\mu_1, \mu_2, \mu_3) = (-10, 0, 6),$$

and

$$\sigma_1^2 = \sigma_2^2 = \sigma_3^2 = 2.$$

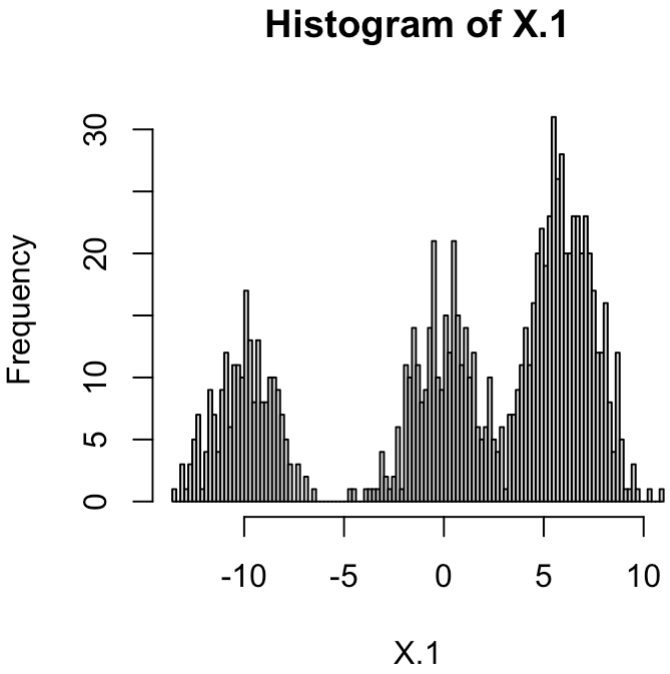
```
set.seed(30027)

# mixture components
mu.true.1     = c(-10, 0, 6)
sigma2.true.1 = c(2, 2, 2)
pi.true.1     = c(0.2, 0.3, 0.5)
NUM.SAMPLES <- 1000

# simulate Z
Z = sample(1:3, NUM.SAMPLES, replace=TRUE, prob = pi.true.1)
```

```
# sample from mixture model
X.1 <- rnorm(NUM.SAMPLES, mean=mu.true.1[Z], sd=sqrt(sigma2.true.1[Z

hist(X.1, breaks=100)
```



Code and notation that we have seen in class

The line

```
Z = sample(1:3, NUM.SAMPLES, replace=TRUE, prob = pi.true.1)
```

implements

$$Z_i \sim \text{Categorical}(\pi_1, \pi_2, \pi_3).$$

Once Z_i has been generated, the line

```
X.1 <- rnorm(NUM.SAMPLES, mean=mu.true.1[Z], sd=sqrt(sigma2.true.1[Z
```

implements

$$X_i \mid Z_i = k \sim N(\mu_k, \sigma_k^2).$$

For example, if $Z_i = 2$, R uses `mu.true.1[2]` and `sigma2.true.1[2]`, so that

$$X_i \mid Z_i = 2 \sim N(0, 2).$$

Example 2: overlapping components

Now simulate 200 samples from

$$Z_i \sim \text{Categorical}(0.2, 0.3, 0.5),$$

with

$$
\begin{aligned}
X_i \mid Z_i = 1 &\sim N(\mu_1 = -2.5, \sigma_1^2 = 2), \\
X_i \mid Z_i = 2 &\sim N(\mu_2 = 0, \sigma_2^2 = 2), \\
X_i \mid Z_i = 3 &\sim N(\mu_3 = 2.5, \sigma_3^2 = 2).
\end{aligned}
$$

The only conceptual difference from the first example is that the component means are much closer together. Therefore, the component distributions overlap much more strongly and it is harder to determine which component generated a particular observation.

```r
set.seed(30027)

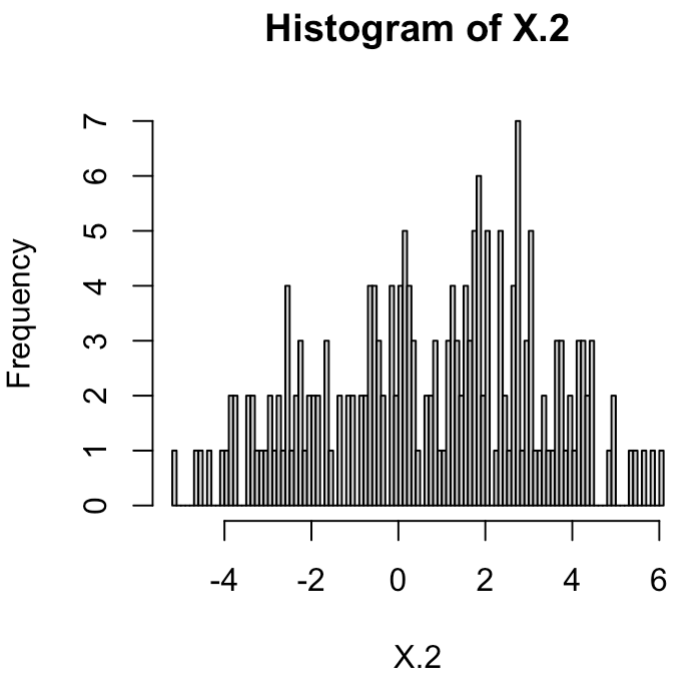
# mixture components
mu.true.2     = c(-2.5, 0, 2.5)
sigma2.true.2 = c(2, 2, 2)
pi.true.2     = c(0.2, 0.3, 0.5)

NUM.SAMPLES <- 200

# simulate Z
Z = sample(1:3, NUM.SAMPLES, replace=TRUE, prob = pi.true.2)

# sample from mixture model
X.2 <- rnorm(NUM.SAMPLES, mean=mu.true.2[Z], sd=sqrt(sigma2.true.2[Z

hist(X.2, breaks=100)
```



Implementation of the EM algorithm for GMM

We now assume that the observed data follow a mixture of three Normal distributions with **known variance** $\sigma^2 = 2$.

For $i = 1, \ldots, n$,

$$Z_i \sim \text{Categorical}(\pi_1, \pi_2, \pi_3),$$

where

$$\pi_1 + \pi_2 + \pi_3 = 1,$$

so that we may equivalently write

$$\pi_3 = 1 - \pi_1 - \pi_2.$$

Conditional on the component,

$$\begin{aligned} X_i \mid Z_i = 1 &\sim N(\mu_1, 2), \\ X_i \mid Z_i = 2 &\sim N(\mu_2, 2), \\ X_i \mid Z_i = 3 &\sim N(\mu_3, 2). \end{aligned}$$

We want to obtain the MLE of

$$\theta = (\pi_1, \pi_2, \mu_1, \mu_2, \mu_3).$$

Although the implementation stores all three mixture proportions, only two of them are free parameters because they must add to one.

The observed-data likelihood

Because we do **not** observe the component labels Z_i, the marginal density of one observation is

$$P(X_i = x_i \mid \theta) = \sum_{k=1}^{3} P(X_i = x_i, Z_i = k \mid \theta).$$

Using

$$P(X_i, Z_i = k \mid \theta) = P(X_i \mid Z_i = k, \theta) P(Z_i = k \mid \theta),$$

we obtain

$$P(X_i = x_i \mid \theta) = \sum_{k=1}^{3} \pi_k f(x_i; \mu_k, 2),$$

where $f(x_i; \mu_k, 2)$ denotes the Normal density with mean μ_k and variance 2.

Therefore, the likelihood is

$$L(\theta) = \prod_{i=1}^{n} \left[\sum_{k=1}^{3} \pi_k f(x_i; \mu_k, 2) \right],$$

and the **incomplete (observed-data) log-likelihood** is

$$\boxed{\ell(\theta) = \sum_{i=1}^{n} \log \left[\sum_{k=1}^{3} \pi_k f(x_i; \mu_k, 2) \right]}.$$

This is the same difficulty we saw in the lecture slides: the sum over the components is **inside the logarithm**, so we cannot simply separate the log-likelihood into one term for each component.

Why introduce the latent labels?

If the labels Z_i were observed, we could write the complete-data likelihood using indicators:

$$P(X, Z \mid \theta) = \prod_{i=1}^{n} \prod_{k=1}^{3} [\pi_k f(x_i; \mu_k, 2)]^{I(Z_i = k)}.$$

The indicator $I(Z_i = k)$ is equal to 1 only for the component that generated observation i, and it is 0 for the other components.

Taking logs gives the complete-data log-likelihood

$$\log P(X, Z \mid \theta) = \sum_{i=1}^{n} \sum_{k=1}^{3} I(Z_i = k) \left[\log(\pi_k) + \log f(x_i; \mu_k, 2) \right].$$

This expression is much easier to maximise. The only problem is that the indicators $I(Z_i = k)$ are unknown because Z_i is latent.

The E-step

Suppose the current parameter values are

$$\theta^0 = (\pi_1^0, \pi_2^0, \pi_3^0, \mu_1^0, \mu_2^0, \mu_3^0).$$

The formal E-step computes

$$Q(\theta, \theta^0) = E_{Z|X,\theta^0} \left[\log P(X, Z \mid \theta) \right].$$

Substituting the complete-data log-likelihood,

$$Q(\theta, \theta^0) = \sum_{i=1}^{n} \sum_{k=1}^{3} E_{Z|X,\theta^0}[I(Z_i = k)] \left[\log(\pi_k) + \log f(x_i; \mu_k, 2) \right].$$

The key identity from class is

$$\boxed{E_{Z|X,\theta^0}[I(Z_i = k)] = P(Z_i = k \mid X_i = x_i, \theta^0)}.$$

Why? Because $I(Z_i = k)$ is a Bernoulli-type indicator: conditional on X and the current parameters, it equals 1 with probability $P(Z_i = k \mid X_i = x_i, \theta^0)$ and 0 otherwise. Therefore its conditional expectation is exactly that probability.

Hence,

$$\boxed{Q(\theta, \theta^0) = \sum_{i=1}^{n} \sum_{k=1}^{3} P(Z_i = k \mid X_i = x_i, \theta^0) \left[\log(\pi_k) + \log f(x_i; \mu_k, 2)\right].}$$

Computing the posterior component probabilities

Using Bayes' rule,

$$P(Z_i = k \mid X_i = x_i, \theta^0) = \frac{P(X_i = x_i \mid Z_i = k, \theta^0) P(Z_i = k \mid \theta^0)}{P(X_i = x_i \mid \theta^0)}.$$

For the GMM,

$$P(X_i = x_i \mid Z_i = k, \theta^0) = f(x_i; \mu_k^0, 2)$$

and

$$P(Z_i = k \mid \theta^0) = \pi_k^0.$$

Therefore,

$$\boxed{P(Z_i = k \mid X_i = x_i, \theta^0) = \frac{\pi_k^0 f(x_i; \mu_k^0, 2)}{\sum_{k'=1}^{3} \pi_{k'}^0 f(x_i; \mu_{k'}^0, 2)}.}$$

For example,

$$P(Z_i = 1 \mid X_i = x_i, \theta^0) = \frac{\pi_1^0 f(x_i; \mu_1^0, 2)}{\pi_1^0 f(x_i; \mu_1^0, 2) + \pi_2^0 f(x_i; \mu_2^0, 2) + \pi_3^0 f(x_i; \mu_3^0, 2)}.$$

For every observation,

$$\sum_{k=1}^{3} P(Z_i = k \mid X_i = x_i, \theta^0) = 1.$$

These are the probabilities that the code stores in `P_ik`.

The M-step

In the M-step, the posterior probabilities calculated in the E-step are treated as fixed numbers, and we maximise

$$Q(\theta, \theta^0)$$

with respect to the parameters θ.

Because the variance is known here, we only update π_k and μ_k.

Update for the means

For component k, differentiate Q with respect to μ_k:

$$\frac{\partial Q(\theta, \theta^0)}{\partial \mu_k} = \sum_{i=1}^{n} P(Z_i = k \mid X_i = x_i, \theta^0) \frac{x_i - \mu_k}{2}.$$

Setting this derivative equal to zero gives

$$\sum_{i=1}^{n} P(Z_i = k \mid X_i = x_i, \theta^0)(x_i - \mu_k) = 0.$$

Therefore,

$$\boxed{\widehat{\mu}_k = \frac{\sum_{i=1}^{n} P(Z_i = k \mid X_i = x_i, \theta^0) x_i}{\sum_{i=1}^{n} P(Z_i = k \mid X_i = x_i, \theta^0)}}.$$

So the new mean is a **weighted mean**, with posterior component probabilities as the weights.

Update for the mixture proportions

The mixture proportions satisfy

$$\pi_1 + \pi_2 + \pi_3 = 1.$$

Maximising the part of Q involving the π_k subject to this constraint gives

$$\boxed{\widehat{\pi}_k = \frac{1}{n} \sum_{i=1}^{n} P(Z_i = k \mid X_i = x_i, \theta^0)}.$$

Thus the updated mixture proportion is the **average posterior membership probability** for component k.

Equivalently, if

$$N_k = \sum_{i=1}^{n} P(Z_i = k \mid X_i = x_i, \theta^0),$$

then

$$\widehat{\pi}_k = \frac{N_k}{n}.$$

N_k can be interpreted as the **effective number of observations assigned to component** k.

EM R implementation

The `mixture.EM` function runs the E-step and M-step repeatedly until the incomplete log-likelihood changes by less than `epsilon`.

```
# pi.init : initial value for pi
# mu.init : initial value for mu
```

```r
# epsilon : If the incomplete log-likelihood has changed by less tha
# EM will stop.
# max.iter : maximum number of EM-iterations
mixture.EM <- function(X, pi.init, mu.init, epsilon=1e-5, max.iter=1

  pi.curr = pi.init
  mu.curr = mu.init

  # store incomplete log-likehoods for each iteration
  log_liks = c()

  # compute incomplete log-likehoods using initial values of paramet
  log_liks = c(log_liks, compute.log.lik(X, pi.curr, mu.curr)$ill)

  # set the change in incomplete log-likelihood with 1
  delta.ll = 1

  # number of iteration
  n.iter = 1

  # If the log-likelihood has changed by less than epsilon, EM will
  while((delta.ll > epsilon) & (n.iter <= max.iter)){

    # run EM step
    EM.out = EM.iter(X, pi.curr, mu.curr)

    # replace the current value with the new parameter estimate
    pi.curr = EM.out$pi.new
    mu.curr = EM.out$mu.new

    # incomplete log-likehoods with new parameter estimate
    log_liks = c(log_liks, compute.log.lik(X, pi.curr, mu.curr)$ill

    # compute the change in incomplete log-likelihood
    delta.ll = log_liks[length(log_liks)] - log_liks[length(log_liks

    # increase the number of iteration
    n.iter = n.iter + 1
  }
  return(list(pi.curr=pi.curr, mu.curr=mu.curr, log_liks=log_liks))
}
```

What is happening inside one EM iteration?

```r
EM.iter <- function(X, pi.curr, mu.curr) {

  # E-step: compute E_{Z|X,theta_0}[I(Z_i = k)]

  # for each sample X_i, compute P(X_i, Z_i=k)
  prob.x.z = compute.prob.x.z(X, pi.curr, mu.curr)$prob.x.z

  # compute P(Z_i=k | X_i)
  P_ik = prob.x.z / rowSums(prob.x.z)

  # M-step
  pi.new = colSums(P_ik)/sum(P_ik)  # sum(P_ik) is equivalent to sa
```

```
  mu.new = colSums(P_ik*X)/colSums(P_ik)

  return(list(pi.new=pi.new, mu.new=mu.new))
}
```

E-step: `prob.x.z`

Inside `compute.prob.x.z`, the code calculates

$$P(X_i = x_i, Z_i = k \mid \theta^0) = \pi_k^0 f(x_i; \mu_k^0, 2).$$

Thus the matrix `prob.x.z` has entries

$$\texttt{prob.x.z}[i,k] = \pi_k^0 f(x_i; \mu_k^0, 2).$$

Each **row** corresponds to one observation i, and each **column** corresponds to one mixture component k.

E-step: `P_ik`

The command

```
P_ik = prob.x.z / rowSums(prob.x.z)
```

divides each joint probability by

$$\sum_{k'=1}^{3} P(X_i = x_i, Z_i = k' \mid \theta^0) = P(X_i = x_i \mid \theta^0).$$

Therefore,

$$\boxed{\texttt{P_ik}[i,k] = P(Z_i = k \mid X_i = x_i, \theta^0)}.$$

This is exactly

$$E_{Z|X,\theta^0}[I(Z_i = k)].$$

So the line of code computing `P_ik` is the computational implementation of the **E-step**.

M-step: `pi.new`

The command

```
pi.new = colSums(P_ik)/sum(P_ik)
```

implements

$$\widehat{\pi}_k = \frac{1}{n} \sum_{i=1}^{n} P(Z_i = k \mid X_i = x_i, \theta^0).$$

Why is `sum(P_ik)` equal to n? For each observation,

$$\sum_{k=1}^{3} P(Z_i = k \mid X_i = x_i, \theta^0) = 1.$$

Therefore,

$$\sum_{i=1}^{n} \sum_{k=1}^{3} P(Z_i = k \mid X_i = x_i, \theta^0) = \sum_{i=1}^{n} 1 = n.$$

Hence `sum(P_ik)` is simply the sample size.

M-step: `mu.new`

The command

```
mu.new = colSums(P_ik*X)/colSums(P_ik)
```

implements

$$\boxed{\widehat{\mu}_k = \frac{\sum_{i=1}^{n} P(Z_i = k \mid X_i = x_i, \theta^0) x_i}{\sum_{i=1}^{n} P(Z_i = k \mid X_i = x_i, \theta^0)}}.$$

Again, each observation contributes to every component, but its contribution is weighted by its posterior probability of belonging to that component.

Computing the incomplete log-likelihood

The function below computes

$$\boxed{\ell(\theta) = \sum_{i=1}^{n} \log \left(\sum_{k=1}^{3} \pi_k f(x_i; \mu_k, 2) \right)}.$$

```
# Compute incomplete log-likehoods
compute.log.lik <- function(X, pi.curr, mu.curr) {

  # for each sample X_i, compute P(X_i, Z_i=k)
  prob.x.z = compute.prob.x.z(X, pi.curr, mu.curr)$prob.x.z

  # incomplete log-likehoods
  ill = sum(log(rowSums(prob.x.z)))

  return(list(ill=ill))
}

# for each sample X_i, compute P(X_i, Z_i=k)
compute.prob.x.z <- function(X, pi.curr, mu.curr) {

  # for each sample X_i, compute P(X_i, Z_i=k).
  # Store these values in the columns of L.
  L = matrix(NA, nrow=length(X), ncol=length(pi.curr))

  for(k in seq_len(ncol(L))) {
    L[, k] = dnorm(X, mean=mu.curr[k], sd=sqrt(2))*pi.curr[k]
  }
```

```
  return(list(prob.x.z=L))
}
```

Connecting `compute.prob.x.z` to probability notation

For each i and k,

```
L[, k] = dnorm(X, mean=mu.curr[k], sd=sqrt(2))*pi.curr[k]
```

computes

$$f(x_i; \mu_k, 2)\pi_k = P(X_i = x_i \mid Z_i = k)P(Z_i = k) = P(X_i = x_i, Z_i = k).$$

Then

```
rowSums(prob.x.z)
```

computes

$$\sum_{k=1}^{3} P(X_i = x_i, Z_i = k) = P(X_i = x_i),$$

by the law of total probability.

Finally,

```
sum(log(rowSums(prob.x.z)))
```

computes

$$\sum_{i=1}^{n} \log P(X_i = x_i \mid \theta) = \ell(\theta).$$

The EM algorithm as implemented here

Starting from initial parameter values θ^0:

1. **E-step:** for every observation and every component calculate

$$P(Z_i = k \mid X_i = x_i, \theta^0) = \frac{\pi_k^0 f(x_i; \mu_k^0, 2)}{\sum_{k'=1}^{3} \pi_{k'}^0 f(x_i; \mu_{k'}^0, 2)}.$$

2. **M-step:** hold those probabilities fixed and calculate

$$\widehat{\pi}_k = \frac{1}{n} \sum_{i=1}^{n} P(Z_i = k \mid X_i = x_i, \theta^0),$$

and

$$\hat{\mu}_k = \frac{\sum_{i=1}^{n} P(Z_i = k \mid X_i = x_i, \theta^0) x_i}{\sum_{i=1}^{n} P(Z_i = k \mid X_i = x_i, \theta^0)}.$$

3. Replace the current parameters by the new estimates,

$$\theta^0 \leftarrow \hat{\theta},$$

and repeat.

4. At each iteration evaluate the incomplete log-likelihood

$$\ell(\theta) = \sum_{i=1}^{n} \log \left[\sum_{k=1}^{3} \pi_k f(x_i; \mu_k, 2) \right].$$

The EM algorithm has the important property that

$$\ell(\theta^{(t+1)}) \geq \ell(\theta^{(t)}).$$

So, apart from numerical round-off, the plotted incomplete log-likelihood should not decrease from one iteration to the next.

Apply the EM algorithm to the first simulated data set

Run the EM algorithm with different initial values and check that the incomplete log-likelihood increases at each step.

Because the three components in the first data set are well separated, we expect the component memberships to be relatively easy to infer and the final parameter estimates to be close to the true values.

```
EM1 <- mixture.EM(
  X.1,
  pi.init=c(0.2,0.3,0.5),
  mu.init=c(-4,1,3),
  epsilon=1e-5,
  max.iter=100
)

ee = EM1

print(paste("Estimate pi = (", round(ee$pi.curr[1],2), ",",
            round(ee$pi.curr[2],2), ",",
            round(ee$pi.curr[3],2), ")", sep=""))
```

```
[1] "Estimate pi = (0.22,0.29,0.49)"
```

```
print(paste("True pi = (", pi.true.1[1], ",", pi.true.1[2], ",", pi
```

```
[1] "True pi = (0.2,0.3,0.5)"
```

```
print(paste("Estimate mu = (", round(ee$mu.curr[1],2), ",",
            round(ee$mu.curr[2],2), ",",
```

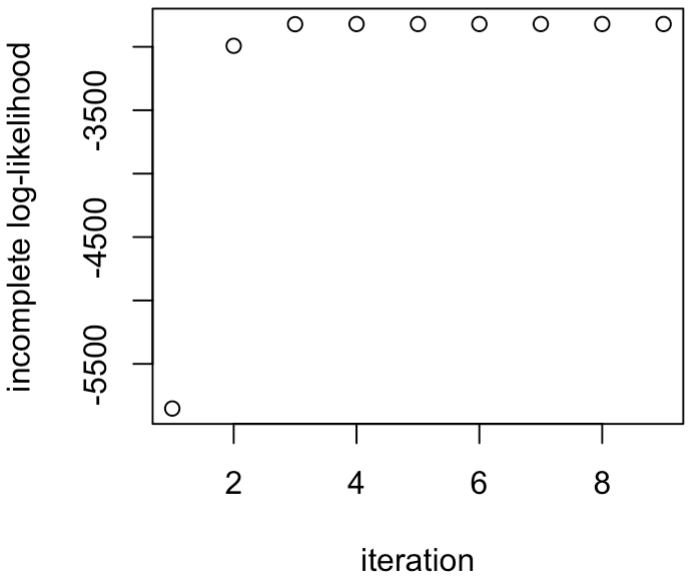
```
            round(ee$mu.curr[3],2), ")", sep=""))
```

```
[1] "Estimate mu = (-10,-0.03,6.06)"
```

```
print(paste("True mu = (", mu.true.1[1], ",", mu.true.1[2], ",", mu.
```

```
[1] "True mu = (-10,0,6)"
```

```
plot(ee$log_liks, ylab='incomplete log-likelihood', xlab='iteration
```




```
EM2 <- mixture.EM(
  X.1,
  pi.init=c(0.9,0.05,0.05),
  mu.init=c(-4,1,3),
  epsilon=1e-5,
  max.iter=100
)

ee = EM2

print(paste("Estimate pi = (", round(ee$pi.curr[1],2), ",",
            round(ee$pi.curr[2],2), ",",
            round(ee$pi.curr[3],2), ")", sep=""))
```

```
[1] "Estimate pi = (0.22,0.29,0.49)"
```

```
print(paste("True pi = (", pi.true.1[1], ",", pi.true.1[2], ",", pi.
```

```
[1] "True pi = (0.2,0.3,0.5)"
```

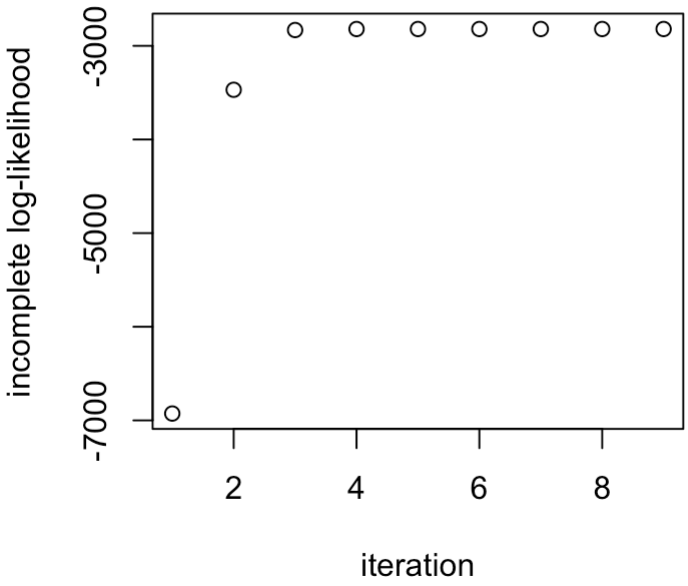
```
print(paste("Estimate mu = (", round(ee$mu.curr[1],2), ",",
            round(ee$mu.curr[2],2), ",",
            round(ee$mu.curr[3],2), ")", sep=""))
```

```
[1] "Estimate mu = (-10,-0.03,6.06)"
```

```
print(paste("True mu = (", mu.true.1[1], ",", mu.true.1[2], ",", mu.
```

```
[1] "True mu = (-10,0,6)"
```

```
plot(ee$log_liks, ylab='incomplete log-likelihood', xlab='iteration
```




```
EM3 <- mixture.EM(
  X.1,
  pi.init=c(0.9,0.05,0.05),
  mu.init=c(10,4,1),
  epsilon=1e-5,
  max.iter=100
)

ee = EM3

print(paste("Estimate pi = (", round(ee$pi.curr[1],2), ",",
            round(ee$pi.curr[2],2), ",",
            round(ee$pi.curr[3],2), ")", sep=""))
```

```
[1] "Estimate pi = (0.49,0.29,0.22)"
```

```
print(paste("True pi = (", pi.true.1[1], ",", pi.true.1[2], ",", pi.
```

```
[1] "True pi = (0.2,0.3,0.5)"
```

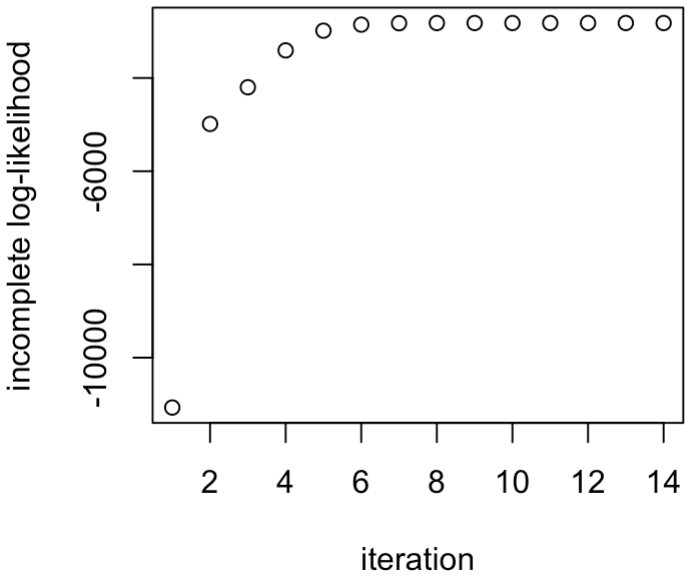
```
print(paste("Estimate mu = (", round(ee$mu.curr[1],2), ",",
            round(ee$mu.curr[2],2), ",",
            round(ee$mu.curr[3],2), ")", sep=""))
```

```
[1] "Estimate mu = (6.06,-0.03,-10)"
```

```
print(paste("True mu = (", mu.true.1[1], ",", mu.true.1[2], ",", mu.
```

```
[1] "True mu = (-10,0,6)"
```

```
plot(ee$log_liks, ylab='incomplete log-likelihood', xlab='iteration
```



Compare the final incomplete log-likelihoods

```
EM1$log_liks[length(EM1$log_liks)]
```

```
[1] -2820.215
```

```
EM2$log_liks[length(EM2$log_liks)]
```

```
[1] -2820.215
```

```
EM3$log_liks[length(EM3$log_liks)]
```

```
[1] -2820.215
```

The estimators from the three EM runs have essentially the same highest incomplete log-likelihood. Some runs may differ only by a permutation of the component labels.

This is called **label switching**. For example, one run may call the component with mean near -10 "component 1", while another run may call exactly the same fitted component "component 2". The fitted mixture distribution is unchanged.

For the first run, the original notes report approximately

$$\widehat{\pi}_1 = 0.2212, \qquad \widehat{\pi}_2 = 0.2854,$$

and

$$\widehat{\mu}_1 = -9.9996, \qquad \widehat{\mu}_2 = -0.0323, \qquad \widehat{\mu}_3 = 6.0559,$$

which are close to the true values.

Apply the EM algorithm to the second simulated data set

Now repeat the procedure for the second data set.

Here the Normal components overlap substantially. Therefore, many observations have posterior probabilities that are not close to 0 or 1. The component assignments are more uncertain, and the likelihood surface can make the algorithm more sensitive to the starting values.

```
EM1 <- mixture.EM(
  X.2,
  pi.init=c(0.2,0.3,0.5),
  mu.init=c(-4,1,3),
  epsilon=1e-5,
  max.iter=100
)

ee = EM1

print(paste("Estimate pi = (", round(ee$pi.curr[1],2), ",",
            round(ee$pi.curr[2],2), ",",
            round(ee$pi.curr[3],2), ")", sep=""))
```

```
[1] "Estimate pi = (0.26,0.34,0.4)"
```

```
print(paste("True pi = (", pi.true.2[1], ",", pi.true.2[2], ",", pi
```

```
[1] "True pi = (0.2,0.3,0.5)"
```

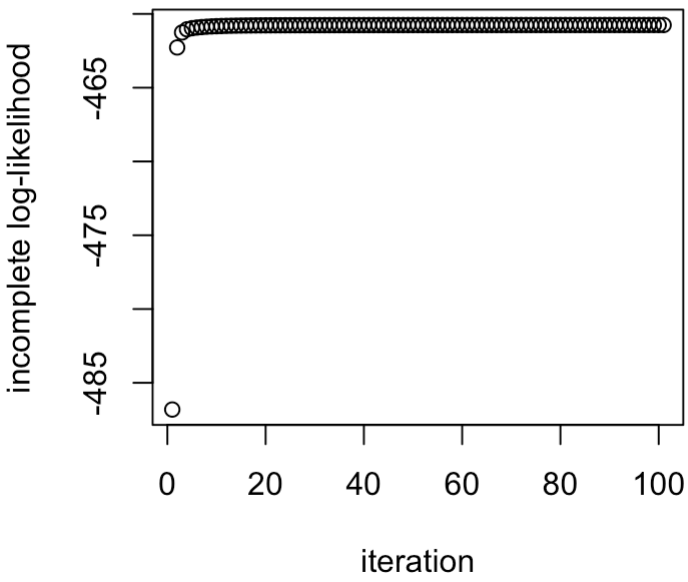
```
print(paste("Estimate mu = (", round(ee$mu.curr[1],2), ",",
            round(ee$mu.curr[2],2), ",",
            round(ee$mu.curr[3],2), ")", sep=""))
```

```
[1] "Estimate mu = (-2.33,0.66,2.91)"
```

```
print(paste("True mu = (", mu.true.2[1], ",", mu.true.2[2], ",", mu
```

```
[1] "True mu = (-2.5,0,2.5)"
```

```
plot(ee$log_liks, ylab='incomplete log-likelihood', xlab='iteration
```




```
EM2 <- mixture.EM(
  X.2,
  pi.init=c(0.9,0.05,0.05),
  mu.init=c(-4,1,3),
  epsilon=1e-5,
  max.iter=100
)

ee = EM2

print(paste("Estimate pi = (", round(ee$pi.curr[1],2), ",",
            round(ee$pi.curr[2],2), ",",
            round(ee$pi.curr[3],2), ")", sep=""))
```

```
[1] "Estimate pi = (0.27,0.35,0.38)"
```

```
print(paste("True pi = (", pi.true.2[1], ",", pi.true.2[2], ",", pi.
```

```
[1] "True pi = (0.2,0.3,0.5)"
```

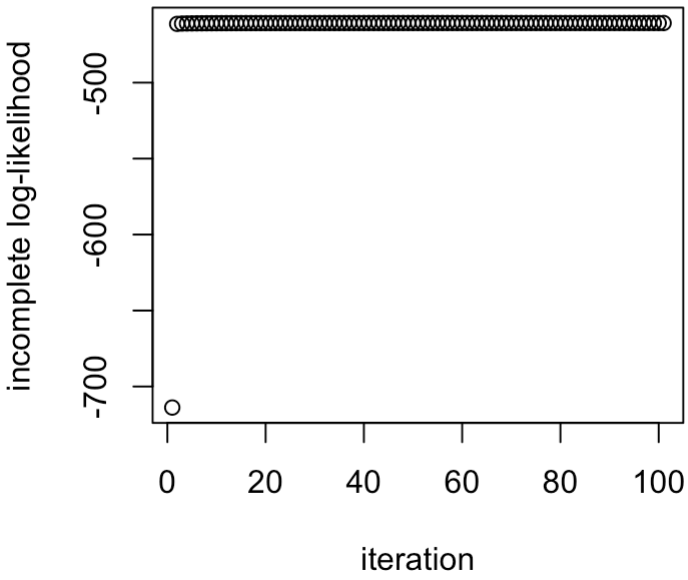
```
print(paste("Estimate mu = (", round(ee$mu.curr[1],2), ",",
            round(ee$mu.curr[2],2), ",",
            round(ee$mu.curr[3],2), ")", sep=""))
```

```
[1] "Estimate mu = (-2.31,0.74,2.94)"
```

```
print(paste("True mu = (", mu.true.2[1], ",", mu.true.2[2], ",", mu.
```

```
[1] "True mu = (-2.5,0,2.5)"
```

```
plot(ee$log_liks, ylab='incomplete log-likelihood', xlab='iteration
```




```
EM3 <- mixture.EM(
  X.2,
  pi.init=c(0.9,0.05,0.05),
  mu.init=c(10,4,1),
  epsilon=1e-5,
  max.iter=100
)

ee = EM3

print(paste("Estimate pi = (", round(ee$pi.curr[1],2), ",",
            round(ee$pi.curr[2],2), ",",
            round(ee$pi.curr[3],2), ")", sep=""))
```

```
[1] "Estimate pi = (0.2,0.49,0.31)"
```

```
print(paste("True pi = (", pi.true.2[1], ",", pi.true.2[2], ",", pi.
```

```
[1] "True pi = (0.2,0.3,0.5)"
```

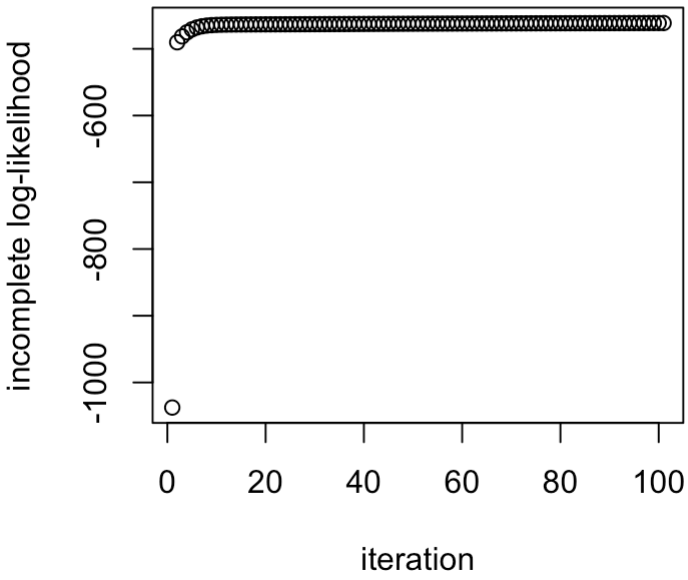
```
print(paste("Estimate mu = (", round(ee$mu.curr[1],2), ",",
            round(ee$mu.curr[2],2), ",",
            round(ee$mu.curr[3],2), ")", sep=""))
```

```
[1] "Estimate mu = (3.46,1.5,-2.11)"
```

```
print(paste("True mu = (", mu.true.2[1], ",", mu.true.2[2], ",", mu.
```

```
[1] "True mu = (-2.5,0,2.5)"
```

```
plot(ee$log_liks, ylab='incomplete log-likelihood', xlab='iteration
```



Compare the final incomplete log-likelihoods

```
EM1$log_liks[length(EM1$log_liks)]
```

```
[1] -460.7521
```

```
EM2$log_liks[length(EM2$log_liks)]
```

```
[1] -460.7533
```

```
EM3$log_liks[length(EM3$log_liks)]
```

```
[1] -461.3282
```

For this data set, the first EM run has the highest incomplete log-likelihood. The original notes report the estimates

$$\widehat{\pi}_1 = 0.2640, \qquad \widehat{\pi}_2 = 0.3363,$$

and

$$\widehat{\mu}_1 = -2.3301, \qquad \widehat{\mu}_2 = 0.6604, \qquad \widehat{\mu}_3 = 2.9052.$$

The fact that different starting values can lead to different fitted solutions is why, in practice, we run EM several times from different initial values and retain the fit with the largest incomplete log-likelihood.

Notes

The central line to remember is

$$\boxed{I(Z_i = k) \quad \longrightarrow \quad E_{Z|X,\theta^0}[I(Z_i = k)] = P(Z_i = k \mid X_i = x_i, \theta^0)}.$$

The complete-data formulas would use the unknown indicators $I(Z_i = k)$. In the E-step, EM replaces those unknown indicators by their conditional expectations. In the M-step, those expectations are treated as fixed weights while the parameters are updated.

In this R file, the connection is particularly direct:

$$\boxed{\texttt{P_ik}[i,k] = P(Z_i = k \mid X_i = x_i, \theta^0) = E_{Z|X,\theta^0}[I(Z_i = k)]}.$$

Then

$$\boxed{\texttt{pi.new}[k] = \widehat{\pi}_k}$$

and

$$\boxed{\texttt{mu.new}[k] = \widehat{\mu}_k}.$$

So the code is not implementing a different version of EM: it is the direct computational translation of the E-step and M-step formulas derived in class.